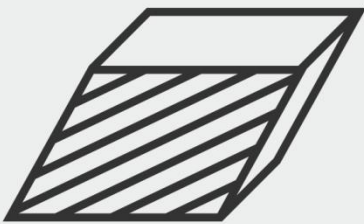




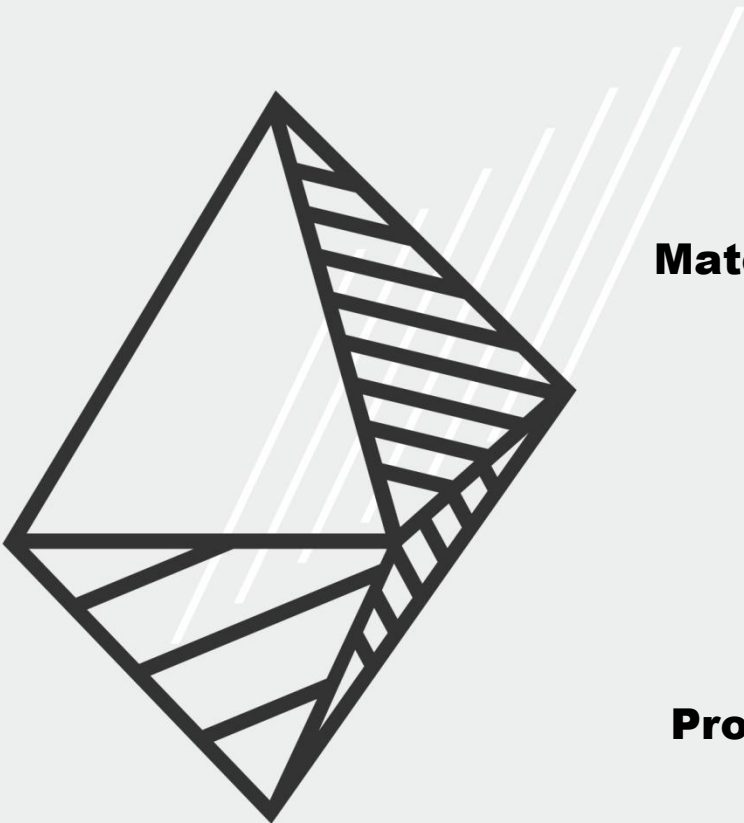
Fecha: 5 de Mayo del 2026



Grupo: 2a

Practica 11

Alumno: Issac Martin Recendiz



**Materia: Programacion Orientada
a Objetos**

Profesor: Jose Jesus Collazo



La herencia es un concepto fundamental en la Programación Orientada a Objetos que permite crear nuevas clases a partir de otras ya existentes. Esto facilita la reutilización de código y ayuda a organizar mejor los programas.

Mediante la herencia, una clase puede adquirir atributos y métodos de otra, lo que evita repetir información y hace que el sistema sea más fácil de mantener. Además, permite que diferentes objetos compartan características comunes, pero al mismo tiempo tengan comportamientos específicos

En esta práctica se aplicará este concepto para diseñar un sistema de personajes de un videojuego, donde todos comparten ciertas características, pero cada uno posee habilidades diferentes.

ANÁLISIS DEL PROBLEMA

El problema consiste en diseñar un sistema para un videojuego en el que existen distintos tipos de personajes. Todos los personajes comparten ciertas características como su nombre, nivel y cantidad de vida.

Sin embargo, cada tipo de personaje tiene habilidades distintas. Por ejemplo, un guerrero se especializa en ataques fuertes, un mago en ataques mágicos y un arquero en ataques a distancia.

Para resolver este problema, es necesario identificar una estructura que permita compartir características comunes y al mismo tiempo diferenciar los comportamientos. Para ello, se utilizará el concepto de herencia.

DISEÑO DE LA SOLUCIÓN

● *Clases identificadas*

- Clase base: **Personaje**
- Clases derivadas:
 - Guerrero
 - Mago
 - Arquero

Además:

- PersonajeEspecial (combinación de habilidades)

● ***Atributos***

Clase Personaje:

- nombre
- nivel
- vida

Clases derivadas:

- Guerrero:
 - fuerza
- Mago:
 - magia
- Arquero:
 - precisión

● ***Métodos***

Clase base (Personaje):

- mostrarInfo() → muestra datos del personaje
- atacar() → acción general de ataque

● **Clases derivadas:**

- Guerrero:
 - atacar() → ataque fuerte
- Mago:
 - atacar() → ataque mágico
- Arquero:
 - atacar() → ataque a distancia

Justificación

Se decidió usar una clase base llamada Personaje porque todos los tipos de personajes comparten características en común. Esto permite evitar repetir código.

Las clases derivadas permiten especializar el comportamiento de cada personaje, principalmente en el método atacar(), ya que cada uno tiene una forma distinta de combatir.

EXPLICACIÓN DE LA HERENCIA

En este sistema, la herencia se aplica haciendo que las clases Guerrero, Mago y Arquero hereden de la clase Personaje.

Esto significa que todas las clases derivadas obtienen automáticamente los atributos nombre, nivel y vida, así como los métodos definidos en la clase base.

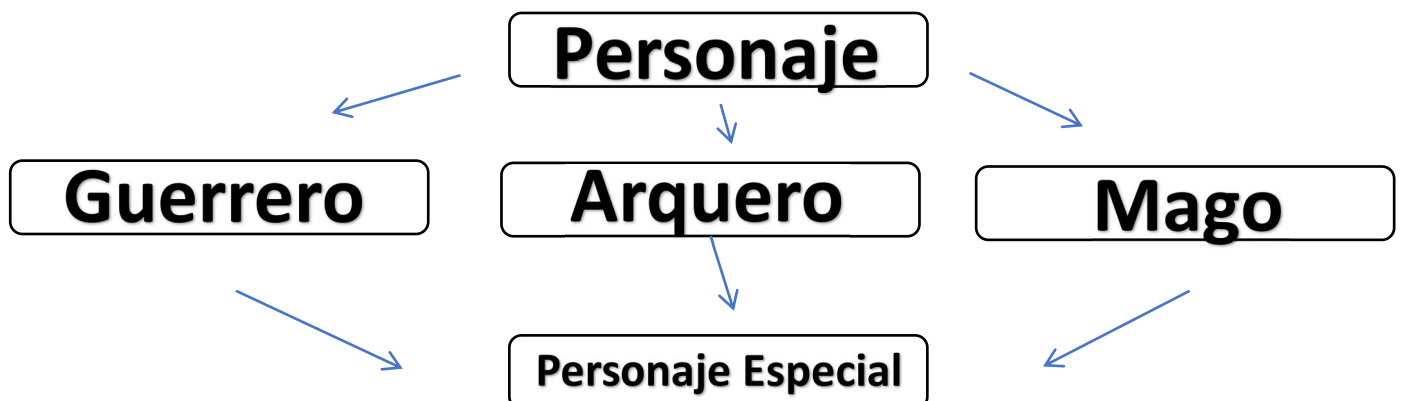
Se eligió esta estructura porque permite mantener un orden claro en el programa, facilitando la reutilización de código y la implementación de nuevos tipos de personajes en el futuro.

PERSONAJE ESPECIAL

Para el personaje especial, se decidió utilizar herencia múltiple, heredando tanto de Guerrero como de Mago.

Esto permite que el personaje tenga habilidades combinadas, como ataques fuertes y ataques mágicos.

La herencia múltiple es útil en este caso porque el personaje especial no pertenece a una sola categoría, sino que combina características de varias.



EXPLICACIÓN PERSONAL

La solución fue organizada de esta manera porque permite separar claramente las características generales de las específicas. Esto hace que el sistema sea más fácil de entender y de ampliar.

Una de las decisiones más importantes fue definir correctamente la clase base, ya que de ella dependen todas las demás. También fue importante decidir cómo implementar el personaje especial, ya que combina características de diferentes clases.

Lo más difícil fue organizar correctamente la relación entre las clases para que tuviera sentido lógico y no se repitiera información.

CONCLUSIÓN

En esta práctica se comprendió la importancia de la herencia dentro de la Programación Orientada a Objetos. Se aprendió cómo estructurar un sistema utilizando clases base y clases derivadas.

Además, se entendió cómo la herencia ayuda a reutilizar código y a crear programas más organizados y eficientes.

Este tipo de diseño es muy útil en el desarrollo de software real, especialmente en videojuegos, donde existen múltiples objetos con características similares pero comportamientos distintos.